Props to Aanand



The demo app used in this chapter was inspired by Aanand Prasad's demo, which shows how to connect a basic Python Flask app to Redis using Compose.[1]

Aanand is the creator of *Fig*—the precursor to Docker Compose—and a former employee of Docker.

Starting a Redis Server

So we want our Rails app to talk to Redis, huh? Well, first we're going to need a Redis server that our application can talk to. As you might expect, we're not going to install and run Redis on our local machine. Instead, let's leverage the power of Docker and start a Redis server inside a container.

Ultimately, we want to add Redis as a new service with Compose. However, since this is the first we've added, we're going to take baby steps. We'll start by seeing how to run Redis in a container using `docker run` before we circle back to get Compose to do this for us automatically. As you gain more experience and confidence, you'll be able to skip this first step and jump straight to setting up a service in Compose.

Using docker run

To start a Redis server with `docker run`, we'd issue the following command:

```
$ docker run --name redis-container redis
```

This command should mostly be familiar: it tells Docker to run a container based on the official Docker `redis` image.[2] However, there are a couple of options we haven't seen before.

Docker gives each new container a unique *container ID* to identify it. However, these long identifiers aren't very human-friendly. Just like when we tagged an image to give it a friendlier name on page 31, the `--name` option tells Docker to give our new container a nice, human-readable name.

Now stop the Redis server by pressing Ctrl-C.

Our ultimate aim is for our `docker-compose.yml` file to fully describe our application, including all its dependencies. Having seen how to start a Redis server with `docker run`, we're ready to set up Compose to manage Redis for us.

1. www.slideshare.net/Docker/compose-breakout-aanand-prasad#8
2. https://hub.docker.com/_/redis/